Proyecto conjunto: metodología y criterios de evaluación

EI1039 – EI1048
Grado en Ingeniería Informática

Curso 2026-2027, versión 1.0

ESTE documento presenta la metodología de desarrollo y los criterios de evaluación del proyecto conjunto de prácticas de las asignaturas *Diseño de Software* (EI1039) y *Paradigmas de Software* (EI1048) para el curso 2026/2027. El proyecto cubre las competencias de ambas asignaturas, que se complementan para la realización de un producto de software de envergadura, que sea funcionalmente operativo, que siga buenas prácticas relativas al diseño de software y de interfaces de usuarios, y que su desarrollo se ajuste rigurosamente a la metodología ágil ATDD.

El proyecto se concibe como una actividad en equipo, afectando en consecuencia a todos los aspectos y tareas atribuibles a la realización del proyecto, desde la comunicación y toma de decisiones consensuada entre los miembros del equipo, hasta el uso de herramientas para el desarrollo y gestión del software de forma colaborativa.

1. Introducción

El proyecto común de las asignaturas EI1039 y EI1048 está diseñado para que el alumnado adquiera las **competencias** de ambas asignaturas. Por lo tanto, las tareas, metodología, herramientas y demás aspectos planteados en este proyecto permiten que el alumnado trabaje los **resultados de aprendizaje** incluidos en ambas asignaturas, siguiendo una metodología de aprendizaje activa dirigida por una evaluación formativa y feedback continuo a lo largo de la ejecución del proyecto en las clases de prácticas. Además, los cinco seminarios propuestos en cada asignatura están estrechamente ligados al desarrollo del proyecto, ya que tres seminarios por asignatura introducen conceptos claves, a su debido tiempo, para que cada equipo los aplique en el desarrollo del proyecto, mientras que los otros dos son conjuntos para ambas asignaturas y se plantean como revisión intermedia del progreso del proyecto.

Las entregas intermedias del proyecto (ver Sección 4) requieren de una calidad aceptable en contenido y presentación. Si alguna entrega no es aceptable, el profesorado la marcará con una **tarjeta amarilla**, es decir, un aviso para que el equipo correspondiente mejore considerablemente su proyecto para la próxima entrega. Si el nivel en la siguiente entrega sigue siendo deficiente, el profesorado la marcará con una **tarjeta roja**, lo cual implica un suspenso automático del proyecto para el equipo en la convocatoria en vigor. Con esto, primamos el trabajo constante de cada equipo a lo largo del semestre.

2. Metodología de desarrollo

El desarrollo del proyecto se realizará según el paradigma ATDD (*Acceptance Test Driven Development*). Si un equipo **no** puede acreditar haber seguido esta metodología para desarrollar el proyecto, **la nota del proyecto será 0**, con independencia de la calidad del trabajo final. *Es decir, el proceso será más importante que el producto final, aunque este último deberá cumplir todos los requisitos funcionales y formativos*.

La definición e implementación de pruebas de integración y aceptación se realizará sobre los componentes de código de más alto nivel de la lógica de negocio de la aplicación, es decir, sobre las clases que ofrezcan servicios

finales en el dominio del usuario. No serán necesarias, por tanto, pruebas de aceptación que involucren interfaces de usuario (la forma de ofrecer un servicio), por la singularidad de las herramientas de automatización y por la fragilidad de las pruebas resultantes.

El desarrollo del proyecto se realizará en equipos de hasta cuatro miembros, siguiendo en lo posible técnicas de programación por pares[1]. Todas las decisiones sobre requisitos, historias de usuario, escenarios y pruebas de aceptación deberéis consensuarlas con los propietarios del producto (el profesorado de prácticas).



Debido a la complejidad del proyecto, y teniendo en cuenta experiencias negativas de cursos anteriores, desaconsejamos la formación de equipos de menos de 3 miembros.

Cómo se ha comentado antes, ATDD es un proceso guiado por pruebas de aceptación. La inferencia de estas pruebas se planteará como sigue:

requisito -> historia(s) de usuario -> escenario(s) -> prueba(s) de aceptación

Más detalladamente, cada *requisito* funcional puede descomponerse en una o varias *historias de usuario*. La descomposición de requisitos en historias de usuario, así como la subdivisión de las segundas en otras más específicas, suele responder a determinados criterios como, por ejemplo, el análisis de escenarios de prueba, roles involucrados, pasos del proceso (*workflow steps*), reglas de negocio (*business rules*), etc. Cada historia de usuario deberá redactarse de acuerdo al patrón estudiado:

Como (rol) quiero (funcionalidad) para (motivación, justificación).

Ejemplo:

Como (usuario) necesito (elegir servicios entre aquellos disponibles) para (consultar exclusivamente información de mi interés).

A su vez, cada *historia de usuario* podrá refinarse en términos de *escenarios* de prueba que permitan describir completamente su alcance funcional, indicando en cada caso la naturaleza válida o inválida del escenario.

En este proyecto te serán dados requisitos e historias de usuario de nivel básico. Así, vuestro proyecto comenzará con la definición de escenarios de prueba.

En el desarrollo de este proyecto, y con el objetivo de mantener bajo control la exploración del espacio funcional, se elegirán dos escenarios relevantes por cada historia, uno válido y otro inválido, a partir de los cuales se inferirán pruebas de aceptación. En caso de que no podáis identificar escenarios inválidos, habladlo con el profesorado de la EI1048. En la definición de escenarios probablemente os surjan dudas sobre el comportamiento del sistema en determinados casos particulares; preguntad al cliente (profesores) y no hagáis suposiciones.

Finalmente, por cada escenario elegido, debéis realizar un breve análisis (textual) del rango de pruebas de aceptación que podrían derivarse, con especial énfasis en casos extremos. A partir de este análisis se formalizará e implementará *una única prueba de aceptación por escenario*. Las pruebas de aceptación deberán respetar el esquema *Given-When-Then* estudiado en clases, y recibir el visto bueno del cliente.

Además de las pruebas de aceptación, la evaluación de servicios que involucren dependencias modeladas mediante interfaces *deberá* incluir una prueba de integración. A continuación se explican las diferencias:

- *Prueba de aceptación*. Prueba sobre una función con sentido para el usuario final (lógica de negocio) que genere una cadena de llamadas sobre componentes operativos e involucre aquellas dependencias externas *reales* requeridas para completar la función (API públicas, formatos de intercambio de datos, base de datos,

[1] Se dedicará un seminario a revisar esta dinámica de trabajo.

etc.). El alcance de estas pruebas suele denominarse "de extremo a extremo" (*end-to-end*). Si un resultado no fuera predecible, una opción poco costosa (aunque también poco informativa) podría ser considerar una prueba exitosa si su ejecución termina sin que se produzca ninguna excepción. Otra alternativa sería comprobar la verosimilitud del resultado, es decir, que este tiene sentido aunque no podamos saber si es correcto (*sanity check*). Veamos un caso práctico. Si no tenemos una expectativa sobre el estado de un sensor exterior de CO2 de una ciudad, podemos comprobar que el valor devuelto está en un dominio de valores más probables, por ejemplo, 300-600 PPM (partes por millón).

- *Prueba de integración*. Prueba sobre una función con sentido para el usuario final (lógica de negocio) en cuya ejecución intervengan dependencias externas. En cada prueba de integración se deberá sustituir (vía inyección de dependencia) *al menos* una de las dependencias reales por un *mock* adecuadamente configurado. Tened en cuenta que tanto los datos de entrada como, potencialmente, el resultado esperado coincidirían exactamente con aquellos de la prueba de aceptación concebida para el mismo escenario.

Habrá funciones (de la lógica de negocio) que se beneficien de ambos tipos de pruebas. Como podréis experimentar, estas serán creadas en momentos distintos del desarrollo y coexistirán en el proyecto final, pues sus objetivos son diferentes y complementarios.

Una vez dispongáis de las historias de usuario, escenarios y de las pruebas de aceptación relacionadas, podréis empezar con el desarrollo del proyecto siguiendo una estrategia incremental por ciclos o iteraciones.

Antes de comenzar a iterar sobre las historias de usuario, se deberá realizar una **iteración 0** con una única *historia de desarrollador* que denominaremos *spike*. Un *spike* es un tipo especial de historia que se utiliza para investigar y entender mejor una tecnología, un requisito, o para aumentar la fiabilidad de futuras estimaciones. El objetivo es mitigar los riesgos críticos del proyecto tan pronto como sea posible. La idea es escribir un número mínimo de líneas de código que nos permita integrar y evaluar las distintas tecnologías involucradas en el proyecto (y demostrar la viabilidad del proyecto o de las tecnologías). El código resultante podría no ser aprovechable para el proyecto. Es decir, la solución del *spike* podría ser un miniproyecto muy simple completamente independiente del proyecto de prácticas, que finalmente termine desechándose. El esfuerzo invertido en este *spike* debería verse compensado de forma notable durante el desarrollo del proyecto.

Una vez concluida la iteración 0, y aprovechando los conocimientos adquiridos, se realizará una planificación inicial en la que distribuiréis las historias de usuario (y sus pruebas) en distintas iteraciones, según prioridad del cliente (profesores) y la complejidad de las historias. La duración de las iteraciones deberá ser aproximadamente igual. Para ello, se recomienda estimar el esfuerzo (horas/equipo) que os supondría el desarrollo de cada historia, a partir de la cantidad y complejidad de los escenarios asociados. Si alguna fuese especialmente costosa o tuviese incertidumbres importantes, debéis valorar su descomposición en historias de menor nivel[2].

Cada iteración comenzará con la *implementación* de todas las pruebas de aceptación de las historias asignadas a esa iteración, para lo cual deberéis diseñar aquellas entidades de *alto* nivel involucradas en las pruebas. Generalmente se tratará de clases proveedoras de servicios que satisfarán directamente las historias de usuario incluidas en la iteración actual, definiendo en cada caso únicamente las signaturas de los métodos públicos requeridos en las pruebas. Con el fin de permitir la ejecución de las pruebas, estos métodos deberán contar con una implementación mínima, sin lógica, que provoque el fallo de la prueba.

Una vez las pruebas hayan fallado contra la implementación sin lógica de los servicios a evaluar, podéis comenzar a escribir progresivamente el código (lógica) que permita a las pruebas pasar gradualmente. Comenzad ordenando las pruebas. Cada una define una miniiteración, en la que tendréis que adoptar las mínimas decisiones de diseño y desarrollo que os permita superar la prueba de interés. Os proponemos seguir el siguiente esquema en cada miniiteración:

1. **Identifica la prueba de aceptación objetivo** que quieres superar en la miniiteración actual.
2. **Ejecuta todas las pruebas escritas hasta ese momento**, incluyendo aquellas relacionadas con ciclos o iteraciones anteriores. Asegúrate que la prueba objetivo falla, al igual que aquellas aún no consideradas.

[2]Se estudiarán estrategias de descomposición de historias.

3. **Escribe *únicamente* el código que permita pasar la prueba objetivo**. Este minidesarrollo generalmente incluirá (re)diseños e implementación (o modificación) de múltiples componentes a distintos niveles de abstracción. En caso de detectar alguna componente con lógica compleja, te proponemos realizar una implementación *test-first*, potencialmente siguiendo el patrón TDD. Te recomendamos documentar este proceso en la memoria/entrega correspondiente.
4. **Si existe dependencia externa, añade una prueba de integración**. Si la implementación incluyese alguna conexión a un servicio externo (API), deberás diseñar e implementar una prueba de integración para el mismo escenario. Esta nueva prueba integrará todas las entidades operativas de la aplicación cliente, pero sustituirá el servicio externo por un *mock* convenientemente configurado a través de la abstracción (interfaz) creada para dicho servicio. Esta prueba deberá crearse y superarse antes de intentar pasar la prueba de aceptación correspondiente, la cual ya sí hará uso de la dependencia real.
5. **Ejecuta *otra vez* todas las pruebas escritas hasta ese momento**, incluyendo aquellas relacionadas con ciclos o iteraciones anteriores. Asegúrate de que la prueba objetivo ahora *sí* pasa, al igual que todas aquellas que ya eran exitosas. Comprueba también que todas las pruebas pendientes continúan fallando.
6. **Refactoriza el código añadido** en esta miniiteración a través de pequeños pasos/cambios. Pasa frecuentemente todas las pruebas para asegurarte que ningún cambio afecta la calidad funcional del código ya lograda. Deberéis tener en cuenta los patrones de diseño que se piden como parte de los requisitos formativos en la Sección 3.2 del documento de descripción del proyecto.
7. **Vuelve al paso 1**, si quedan pruebas de aceptación no consideradas.

Al terminar el proceso anterior deberíais contar con un software con funcionalidad parcial validada, según el ciclo actual de desarrollo, que pasa todas las pruebas asociadas a las historias resueltas hasta ese momento.

Este pequeño vídeo[3] os ayudará a comprender la filosofía de trabajo que encierra TDD, mientras que este otro vídeo[4] os mostrará el valor de las historias de usuario como piezas para organizar el desarrollo.

Como resumen de lo estudiado en las clases de teoría, tened en cuenta que las historias de usuario que elaboréis deberían cumplir, idealmente, las siguientes características (criterios **INVEST**):

Independientes. Las historias de usuario no deben depender unas de otras; se debe poder trabajar con ellas por separado. Esto evita dependencias en la planificación y dificultades en la estimación del esfuerzo.

Negociables. Una historia de usuario no es un contrato, es un recordatorio. Debe recoger cuantos más detalles mejor; no obstante, si alguno no está presente no quiere decir que no se deba tener en cuenta.

Valiosas para el cliente. Las historias deben tener sentido para el cliente (resolver una necesidad). Una historia como *Desplegar la aplicación en un clúster de servidores* podría no ser de interés para el cliente.

Estimables. La unidad de medida del esfuerzo la define el grupo de trabajo. Pueden ser horas de programador, días de equipo, etc.

(Small) Pequeñas. El tamaño de una historia debe de ser relativamente pequeño, de forma que su implementación pueda ser ágil e incluirse cómodamente en una iteración.

(Testable) Validable. Los criterios de validación de una historia deberían quedar claros en su redacción, y dar lugar a una serie de pruebas de aceptación que permitan evaluar la funcionalidad recogida en la historia.

También debéis tener en cuenta lo siguiente:

- Las pruebas de aceptación sirven para validar vuestro trabajo, pero la última palabra siempre la tiene el cliente —este vídeo[5] os lo cuenta mejor.
- Las historias de usuario pueden cambiar durante el desarrollo del proyecto. Es posible que el cliente añada,

[3] https://www.youtube.com/watch?v=uGaNkTahrIw
[4] https://www.youtube.com/watch?v=LGeDZmrWwsw&list=PL1CCC9816DB553DDC&index=22
[5] https://www.youtube.com/watch?v=9FnZsaRujpw

modifique, o elimine historias de usuario a medida que el proyecto avanza. E incluso, como hemos visto en clases, podrían surgir nuevas historias a partir del análisis de las existentes.

- Antes de iniciar el trabajo sobre una historia de usuario es necesario que valoréis el esfuerzo que os puede llevar su desarrollo —este otro vídeo[6] muestra una técnica para la estimación del tiempo de desarrollo.
- Como resultado del proyecto deberías obtener pruebas de unidad (deseables, aunque opcionales), de integración y de aceptación.

3. Trabajo en equipo y desarrollo colaborativo

En una primera fase os proponemos que trabajéis de forma conjunta definiendo escenarios, pruebas de aceptación, tomando decisiones de diseño de alto nivel como, por ejemplo, la elección de la arquitectura software, y realizando una planificación inicial, es decir, estimación de historias y distribución de estas en ciclos/iteraciones. En definitiva, es importante en esta primera fase obtener una visión compartida del proyecto y resolver malentendidos y/o ambigüedades entre los miembros del equipo con respecto a los requisitos del proyecto.

En una segunda fase, durante el desarrollo del proyecto, se pondrán en práctica metodologías de desarrollo colaborativo utilizadas en entornos profesionales. El eje central de estas metodologías es el uso de un repositorio centralizado en la nube con un sistema de control de cambios (VCS), el cual os permitirá trabajar de forma independiente en entornos locales para posteriormente integrar vuestros avances en el repositorio centralizado. Este repositorio deberá incluir el código operativo y el código de pruebas, debidamente separados. Os recomendamos encarecidamente usar Git como VCS de la mano de GitHub como *host* del repositorio en la nube.

Cuando varios desarrolladores trabajan con un repositorio centralizado con VCS, es posible que aparezcan problemas (*merge conflict*) a la hora de integrar los cambios de cada uno de ellos si no se toman las medidas adecuadas y si no existe una coordinación muy estrecha entre los miembros del equipo. Como es lógico, esto no siempre es fácil, sobre todo en equipos formados por un gran número de desarrolladores, donde cada individuo tiene su forma de trabajar. Es decir, si empleamos en nuestro equipo un VCS sin tener presente una metodología de colaboración concreta y con cada desarrollador trabajando a su manera, el caos está garantizado. Existen varias metodologías que describen como emplear correctamente estos sistemas, algunos complejos como Gitflow[7], y otros más simples como GitHub flow[8].

En nuestro caso, utilizaremos una metodología adaptada, inspirada tanto en *Gitflow* como en *GitHub flow*. La Figura 1 muestra un diagrama de esta metodología, cuyas características son:

- La rama principal (`main`): los cambios que realizan los desarrolladores no se aplican sobre esta rama directamente, sino que se aplicarán fusionando otras ramas. Debe contener versiones estables del proyecto.
- Las ramas de iteración (`it-`*`nn`*): por cada iteración del desarrollo, se creará una de estas ramas, donde *`nn`* será el número de la iteración. Por ejemplo, la rama de la tercera iteración será `it-03`. Esta rama servirá de punto de partida y de fin para las ramas de historia. Una vez todas las ramas de historia de la iteración se hayan fusionado, se abrirá la *pull request* `IT`*`nn`*, donde se revisará que todos los cambios introducidos son estables. Una vez hechas las comprobaciones, la rama de iteración se fusionará con la rama principal.
- Las ramas de historia (`h-`*`xx`*): por cada historia de la iteración *`nn`*, se creará una de estas ramas, donde *`xx`* será el número de la historia. Por ejemplo, la rama de la quinta historia será `h-05`. En esta rama se incluirán *commits* con los cambios que se hayan ido desarrollando para implementar la historia `xx`. Por ejemplo, el primer *commit* en una de estas ramas podría ser el que implementa las pruebas de aceptación, y los siguientes los que implementan la funcionalidad que pasa las pruebas. Una vez la historia se haya completado, se abrirá la *pull request* `IT`*`nn`*`-H`*`xx`*, donde los miembros del equipo revisarán los cambios

[6] https://www.youtube.com/watch?v=0FbnCWWg_NY
[7] https://nvie.com/posts/a-successful-git-branching-model/
[8] https://docs.github.com/es/get-started/quickstart/github-flow

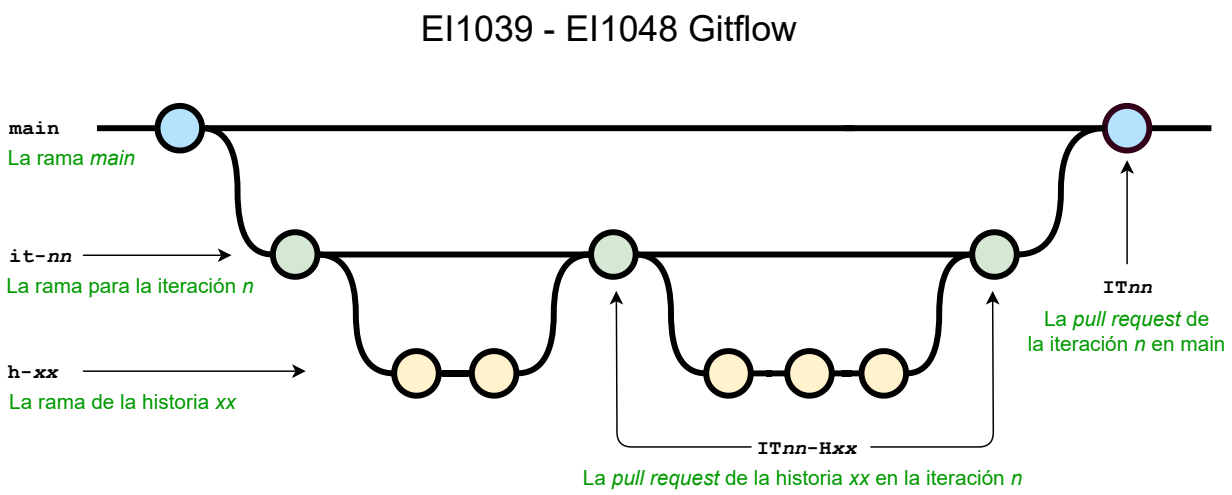


Figura 1: Diagrama de la metodología de uso de Git.

implementados. Si todo es correcto, la *pull request* se aprobará y se fusionará con la rama de la iteración `nn`.



Los nombres propuestos para las ramas (`it-nn`, `h-xx`) y las *pull requests* (`ITnn`, `ITnn-Hxx`) son orientativos. El principal objectivo es que quede claro tanto para vosotros como para el profesor de prácticas a que historia/iteración pertenece cada rama y *pull request*.

Esta metodología permite que los integrantes del grupo puedan trabajar de forma paralela en varias historias, de forma que los cambios introducidos en una historia no afecten a otras, minimizando los conflictos durante su desarrollo. Una vez finalizada una historia, se fusionará con la rama de iteración, donde (probablemente) deberán resolverse conflictos provocados por los cambios introducidos por otras ramas de historia previamente fusionadas. Esta metodología no hace desaparecer los conflictos, sino que los minimiza y los traslada a un punto donde su resolución sea más sencilla. Es recomendable que el equipo realice reuniones periódicas de coordinación, así como para realizar las fusiones de las ramas y resolver los conflictos.

La Figura 2 muestra y describe un ejemplo de desarrollo con esta metodología:

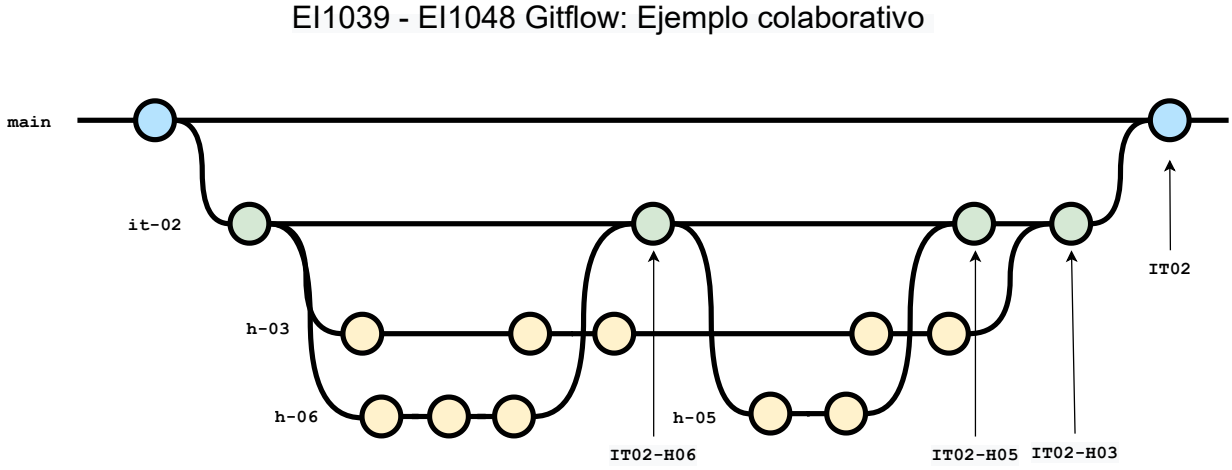


Figura 2: Ejemplo de desarrollo con la metodología descrita

- La rama *main* contiene las funcionalidades implementadas en la primera iteración.

- Se empieza a trabajar en la segunda iteración. Se crea la rama de iteración `it-02`.
- La iteración tiene tres historias: H3, H5 y H6. El equipo empieza a trabajar en paralelo con las historias H3 y H6. Por tanto, se crean las ramas `h-03` y `h-06`.
- Finaliza la implementación de la historia H6. Se crea la *pull request* `IT02-H06`. El equipo revisa los cambios introducidos, comprueban que las pruebas pasan y fusionan la rama con la rama de iteración.
- El desarrollo de la historia H3 sigue su curso, paralelamente se inicia la historia H5. Se crea la rama `h-05`.
- Finaliza la implementación de la historia H5. Se crea la *pull request* `IT02-H05`. El equipo revisa los cambios introducidos, comprueban que las pruebas pasan y fusionan la rama con la rama de iteración.
- Finaliza la implementación de la historia H3. Se crea la *pull request* `IT02-H03`. El equipo revisa los cambios introducidos y comprueban que hay conflictos con algunos cambios introducidos por las historias H5 y H6. Se resuelven los conflictos, comprueban que las pruebas pasan y fusionan la rama con la rama de iteración.
- La iteración ha finalizado. Se crea la *pull request* `IT02` y el equipo revisa los cambios introducidos, la estabilidad de las funcionalidades y comprueban que las pruebas pasan. Finalmente se fusiona la rama de iteración con la rama principal.

Otra parte importante del desarrollo cooperativo es que los miembros del equipo empleen un lenguaje común. No nos referimos al lenguaje de programación (obviamente, si un miembro del equipo desarrolla con Java y otros en C++, hay algo que está fallando), si no al lenguaje que se emplea en los *commits*. Es decir, ¿cómo describimos los cambios que se incluyen en los *commits*? Si cada miembro del equipo utiliza un lenguaje, forma, y semántica distinta, entonces el caos está servido. Seguir un estándar a la hora de definir los mensajes descriptivos de los *commits* es tan o más importante que nombrar correctamente clases, métodos, variables. De hecho, los *commits* son una forma de documentación: son un registro del progreso del desarrollo en el tiempo. Por lo tanto, durante el desarrollo del proyecto usaremos, **como parte de la metodología colaborativa para la realización del proyecto**, una especificación de *commits* basada en Conventional Commits[9], adaptada a nuestras necesidades:

<type>(<scope>): <description>

- `<type>`: indica el tipo de cambio que incluye el *commit*. Usaremos los siguientes:
  - `test`: cambios relacionados con las pruebas.
  - `feat`: cambios que introducen una nueva funcionalidad (*feature*) al proyecto.
  - `refactor`: cambios que introducen modificaciones para optimizar el diseño de funcionalidades previamente implementadas.
  - `fix`: cambios para arreglar *bugs*.
  - `chore`: cambios relacionados con la configuración del proyecto. Por ejemplo, añadir/modificar dependencias.
- `<scope>`: indica donde se producen los cambios (*core*, *ui*, ...). En nuestro caso lo vamos a usar para indicar a que iteración e historia pertenece el *commit*. Lo indicaremos de la siguiente forma: `it`*`nn`*`h`*`xx`*, donde *`nn`* será en número de iteración y *`xx`* el número de historia.
- `<description>`: breve descripción de los cambios introducidos en el *commit*, en inglés. Intentad utilizar descripciones cortas y concisas, pero que aporten información (nada de *'bug fix'* o *'some changes'* con cambios en 30 ficheros...). Si no sois capaces de describir los cambios que incluye el *commit* de forma concisa y breve es posible que necesitéis separar los cambios en varios *commits*.

Un ejemplo de descripción de un *commit* relativo a la creación de los test de aceptación para la primera iteración de la historia de usuario 02 sería:

[9] https://www.conventionalcommits.org/en/v1.0.0/

test(it01h02): create acceptance tests



En la especificación de *commits* el elemento `<type>` suele ser flexible y lo normal que se adapte a las necesidades el equipo de desarrollo. Es decir, se pueden añadir o eliminar tipos si el equipo lo considera necesario.

4. Entregas: contenido y fechas

Se realizará una evaluación continua del progreso del proyecto en las sesiones de prácticas, en dos **seminarios conjuntos** de las dos asignaturas dedicados al proyecto y, a través de **tres informes** que deberéis presentar dentro de los plazos previstos.

Tanto los informes como los seminarios conjuntos dedicados al progreso del proyecto se rigen por un sistema de tarjetas amarillas/rojas. La falta de calidad en el contenido y/o en la presentación de un informe (o de las actividades planteadas en los seminarios conjuntos) será penalizada con una tarjeta amarilla. Aquellos equipos que obtengan una tarjeta amarilla en una entrega (o seminario), deberán subsanar las deficiencias antes de la siguiente entrega, consensuando los plazos con el profesorado de prácticas. La reiteración de deficiencias en la siguiente entrega supondrá una tarjeta roja, y el suspenso del proyecto para todos los miembros del equipo.

En el caso de que algún informe se entregue *fuera* de su plazo ordinario, se penalizará la nota final del proyecto con un 10 %. Un segundo retraso en la entrega de un informe supondrá automáticamente el suspenso del proyecto para todos los miembros del equipo.

La temporización y contenido de cada informe y de los dos seminarios conjuntos se describe como sigue:

- **Entrega 1**. Breve informe con los siguientes apartados. Fecha límite: **viernes 16 de octubre**.
  - Listado de requisitos e historias de usuario que se prevén implementar. Análisis de escenarios y pruebas de aceptación asociadas. Cada historia deberá cumplir criterios **INVEST**, cada escenario deberá incluir un breve comentario que explique su tipo (válido o inválido) y su necesidad, y cada prueba de aceptación se ajustará al esquema *Given-When-Then*. Cualquier decisión importante deberá aparecer debidamente razonada.
  - El código funcional de la iteración 0 (*spike*). Se proporcionará la URL de acceso al repositorio del miniproyecto del *spike* (si es privado, por favor, deberéis añadir al profesorado como colaborador).
  - Propuesta de diseño de interfaces gráficas de usuario para cada una de las historias de usuario definidas previamente. No se requieren interfaces de usuario definitivas, sino más bien *mock-ups* que ayuden a entender visualmente cada una de las historias de usuario. Por lo tanto, los escenarios de cada historia de usuario deberán ser congruentes con las interfaces de usuario planteadas. Cualquier decisión importante deberá aparecer debidamente razonada. Este apartado será revisado por cada equipo en la Entrega 3 (final) a medida que el proyecto evolucione hacia un estado definitivo.
  - Documentación de experiencias con IA generativa, indicando en cada caso la tarea afectada (HU, pruebas de aceptación, *spike*, etc.), la secuencia de interacciones (contextos + preguntas), análisis crítico de las respuestas incluyendo beneficios y limitaciones, y recomendaciones finales.
- **Seminario conjunto 1**. Presentación del equipo con los siguientes apartados. Fecha: **jueves/viernes 22/23 de octubre**.
  - Descripción de las tecnologías seleccionadas para la realización del proyecto (*tech stack*), incluyendo las herramientas de colaboración y gestión del proyecto. Sigue la regla de *keep it simple*, ya que el número tecnologías distintas no es garantía del éxito del proyecto.
  - Reflexión grupal de los riesgos (falta de formación en la tecnología, etc.) y de un plan de contingencia. En particular, cada equipo debe identificar una serie de riesgos potenciales, lo más concretos posibles,

en el corto plazo (que puedan suceder antes de la Entrega 2). Para cada riesgo, plantear un plan/acción de prevención (para que no sucede), y un plan de contingencia (si sucede).

- Presentación del *spike* funcional incluido en la Entrega 1. Por ejemplo, un vídeo demo de la ejecución del *spike* es aceptable. Este vídeo debe mostrar que el equipo ha testeado la tecnología seleccionada de forma adecuada. Esto incluye tanto la demostración de la comunicación entre un cliente y los servicios remotos y/servicios de bases de datos, como la implementación de una prueba (*test*) piloto.
- Breve comentario de la estrategia para el análisis de las historias de usuario descritas en la Entrega 1. No se pide explicar cada una, sino el análisis llevado a cabo para desglosar cada historia en escenarios y cómo se ha presentado en el documento de la Entrega 1.
- Si procede, discusión de experiencias en el uso de IA generativa.

- **Entrega 2**. Breve informe con los siguientes apartados. Fecha límite: **viernes 13 de noviembre**.
  - Planificación consensuada de la distribución razonada de historias en iteraciones, revisadas previamente con el profesorado de prácticas en las sesiones de prácticas.
  - Código de las pruebas de aceptación correspondientes a las historias de la iteración 1.
  - Diagrama de clases UML del sistema bajo test de la iteración 1 para poder ejecutar las pruebas anteriores.
  - URL de acceso al repositorio del proyecto principal (si es privado, por favor, recordad añadir al profesorado de prácticas). Este repositorio incluirá el código ejecutable de las pruebas, junto a la implementación mínima de las clases y métodos *necesarios* para la ejecutar las pruebas.
  - Justificación del patrón de diseño utilizado/pensado para la interfaz de usuario (MVC, MVVC, etc), según la tecnología empleada y criterio arquitectural de vuestro proyecto. ¿En qué partes del proyecto tiene sentido? ¿Se utilizan varios patrones de forma mixta en el proyecto? ¿Por qué es más idónea vuestra elección con respecto a otros patrones de diseño?
  - Análisis preliminar de los patrones de diseño seleccionados (*factory*, etc.) que se prevé que se podrían utilizar. Se deben seleccionar al menos 5 patrones de diseño. Como parte del análisis, se deben proporcionar los diagramas de clases de los patrones de diseño previstos, junto al diagrama de clases de la solución conceptual/genérica de cada patrón (vistos en clase de teoría).
  - Documentación de experiencias con IA generativa, indicando en cada caso la tarea afectada (patrones de diseño, pruebas de integración con *mocks*, lenguajes de programación, etc.), la secuencia de interacciones (contextos + preguntas), análisis crítico de las respuestas incluyendo beneficios y limitaciones, y recomendaciones finales.

  En esta entrega quedará clara vuestra decisión de añadir o no aspectos adicionales no funcionales (ver Sección 3.3 del documento de descripción del proyecto) al proyecto. Si es así, debéis considerar igualmente todos los requisitos del proyecto, y consensuar con el profesorado de prácticas los aspectos adicionales que vais a abordar.

- **Seminario conjunto 2**. Presentación del equipo con los siguientes apartados. Fecha: **jueves/viernes 19/20 de noviembre**.
  - Planificación de las iteraciones para las historias de usuario. Comentar la estimación de la dificultad de las iteraciones y la discrepancia, si existe, con el coste real. Indicar la iteración (o interacciones) completada(s) en el sentido de la codificación de las pruebas de aceptación. Como mínimo, como se exige en la Entrega 2, la iteración 1 debe estar completada. Si fuera el caso, indicar la iteración (o interacciones) completada(s) en cuanto a la implementación final (para la demo).
  - Breve explicación del código de alguna prueba de aceptación correspondientes a las historias de usuario de la iteración 1.
  - Demo de lo implementado hasta la fecha, pero que sea coherente con la planificación temporal de las iteraciones en la Entrega 2.

  - Actividad de contribuciones en GitHub (*commits*, *issues*, etc) en forma de gráfico de ramas (Gitflow, etc) y que explican asimetrías de actividad (es decir, de contribución y/o participación al proyecto), si hay, entre los miembros del equipo.
  - Resumen de la justificación del patrón de diseño para la interfaz de usuario.
  - Resumen del análisis preliminar e idoneidad de los patrones de diseños seleccionados.
  - Breve comentario sobre si alguno de los riesgos identificados en el Seminario conjunto 1 ha surgido y, en consecuencia, si se ha aplicadao el plan de contingencia correspondiente. O por el contrario, si el plan de prevención ha sido eficaz.
  - Si procede, discusión de experiencias en el uso de IA generativa.

- **Entrega 3**. Informe final con los siguientes apartados. La sección 7 incluye recomendaciones sobre las secciones que el informe final debe contener, así como la longitud máxima del informe. Fecha límite: **5 Enero 2027, 23:55 horas**.

  - Resultados por iteraciones: historias terminadas, pruebas implementadas (integración, aceptación), de ellas cuáles pasaron y cuáles no, distribución del trabajo entre miembros del equipo, y si los resultados dieron lugar a modificaciones en la planificación inicial.
  - Listado de todas las pruebas de *aceptación* implementadas, justificación en cada caso, y breve explicación sobre su funcionamiento.
  - Listado de todas las pruebas de *integración* implementadas, justificación en cada caso, y breve explicación sobre su funcionamiento con especial énfasis en la necesidad y configuración de los *mocks* utilizados.
  - Implementación definitiva del patrón de diseño utilizado en el desarrollo de las interfaces gráficas de usuario, justificando su grado de usabilidad. Actualizar adecuadamente el contenido de lo expuesto anteriormente en la Entrega 2.
  - Implementación definitiva de los patrones de diseño finalmente implementados, justificando su idoneidad para el proyecto. Actualizar adecuadamente el contenido de lo expuesto anteriormente en la Entrega 2.
  - Análisis crítico del diseño definitivo de la *interfaz gráfica de usuario*, a partir de la propuesta presentada en la Entrega 1. Justificar los cambios y mejoras llevadas a cabo con respecto a lo expuesto en la Entrega 1. Se deben incluir en este informe tanto las interfaces *mock-ups* (Entrega 1) como las interfaces definitivas para apreciar y valorar la evolución entre entregas.
  - Conclusiones, incluyendo el grado de cumplimiento de los objetivos y tu experiencia personal en el desarrollo del proyecto.

  La corrección de la entrega final incluirá la construcción del proyecto, la ejecución de todas las pruebas implementadas, la ejecución de las aplicaciones para evaluar su alcance funcional y la evaluación del informe. Cada equipo será convocado a una tutoría obligatoria en la que tendréis que realizar una defensa oral del proyecto única para ambas asignaturas. El tribunal está compuesto por el profesorado de ambas asignaturas.

5. Criterios de evaluación

Para concluir esta primera parte, debéis tener en cuenta los siguientes criterios de evaluación:

a) Calidad del producto de software final, considerando alcance (cantidad de funcionalidad conseguida), tecnologías usadas, usabilidad, estética, etc., hasta 2 puntos de la nota final.

b) Historias de usuario, escenarios de prueba, definición de pruebas de aceptación e integración, implementación de las pruebas, uso y diseño de mocks, hasta 3 puntos de la nota final.

c) Implementación de los patrones de diseño, idoneidad de las soluciones de diseño propuestas, justificación del patrón de diseño utilizado en el desarrollo de las interfaces gráficas de usuario, y grado de cumplimiento del objetivo de arquitectura desacoplada, hasta 3 puntos en la nota final.

d) Informes entregados y presentación (calidad, entrega en plazo, etc.), hasta 2 puntos de la nota final. Se valorarán positivamente informes con estructura adecuada, estilo directo y argumentos precisos, según las recomendaciones de la Sección 7. Por el contrario, se valorarán de forma negativa informes innecesariamente extensos, sin estructura y ambiguos.

La nota resultante de los cuatro criterios anteriores, determinados por los requisitos descritos en la Sección 3.1 del documento de descripción del proyecto, se escalará de forma que la nota máxima sea un 8. Los dos puntos restantes para llegar al 10 se otorgarán en función del número y calidad de los aspectos adicionales implementados (ver Sección 3.3 del documento de descripción del proyecto).



Cuando se trabaja en equipo se espera que el reparto de tareas sea equitativo entre todos los miembros. La nota final del proyecto se aplicará, *por defecto*, a todos los miembros por igual.
Sin embargo, el profesorado tomaría medidas oportunas en el caso de una dejadez o falta de contribución significativa por parte de algún miembro, bien observada por otros miembros del equipo o por el profesorado, llegando incluso al bloqueo en el desarrollo del proyecto.
Estas medidas pueden incluir desde una reducción de la nota del proyecto al miembro que no ha cumplido con sus tareas hasta el suspenso del proyecto para ese miembro.



Si se incurre en cualquiera de las siguientes situaciones, **la nota final del proyecto para ambas asignaturas será 0**:

- La aplicación **no funciona** durante la defensa del proyecto.
- No se demuestra haber seguido la metodología **ATDD** durante el desarrollo.
- No se alcanza el número mínimo de **patrones de diseño** o si su uso no esta justificado.
- Durante las entregas parciales se ha obtenido una **tarjeta roja**.

6. Política de uso de IA generativa

La política de uso de herramientas de IA generativa se comentará en el primer seminario de la asignatura EI1048, donde se profundiza en la descripción del proyecto común.

En general, el uso de chatGPT (o herramientas similares) no será penalizado si se utiliza para optimizar o adelantar trabajo en el seno de cada grupo. El profesorado es consciente de que se trata de una tecnología disruptiva de aplicación en muchos sectores y ámbitos del conocimiento, especialmente en la programación y desarrollo de software. Sin embargo, no están tan claras las IMPLICACIONES de su uso en cuanto a aprendizaje de conocimiento y adquisición de competencias por parte de sus usuarios. Por lo tanto, fomentamos el uso de este tipo de herramientas pero con responsabilidad y sentido crítico, siempre que se cumplan las siguientes condiciones:

- Transparencia
- Honestidad
- Uso responsable
- Documentación de interacciones (contextos + preguntas) para entender cómo se desenvuelve una IA generativa en tareas de ingeniería de software (HU, patrones de diseño, pruebas de aceptación, etc.).
- Análisis crítico de respuestas que permita identificar beneficios y limitaciones en cada caso.

- Se penalizarán evidencias de uso de IA generativa no documentadas, o usos *acríticos* de respuestas automáticas que conduzcan a soluciones inadecuadas, ineficientes o incorrectas.

7. Informe: estructura y extensión

La organización de las secciones que se recomienda a continuación se refiere al último informe final (Entrega 3). Las dos primeras entregas son de formato libre.

1. **Introducción**: Debe incluir los objetivos del proyecto así como los enlaces pertinentes al repositorio o repositorios del proyecto desarrollado. Si un equipo decide utilizar repositorios distintos para el *frontend*, *backend*, *spike*, etc., se deben incluir los enlaces para cada uno de los repositorios, indicando el identificador del último *commit* realizado antes de la entrega final.
2. **Requisitos**
   - Listado de requisitos e historias de usuario. Para cada historia de usuario, incluir escenarios y el análisis de las pruebas de aceptación pertinentes por escenario. Incluir también los tests de integración y aceptación, preferiblemente mediantes enlaces al código correspondiente en el repositorio.
3. **Planificación del proyecto**
   - Spike o Iteración 0. Explicar objetivos, logros, decisiones derivadas de este ejercicio, etc.
   - Planificación de historias de usuario en iteraciones. Indicar las historias, pruebas de aceptación, etc. por cada iteración. Indicar el miembro del equipo responsable de cada historia.
4. **Diseño**
   - Tecnologías utilizadas. Breve descripción de las tecnologías empleadas en el proyecto, dependencias, versiones, etc.
   - Diagrama UML de clases del proyecto.
   - Patrón o patrones de diseño de la interfaz gráfica.
   - Evolución de la interfaz gráfica por iteraciones.
   - Patrones de diseño.
5. **Aspectos adicionales** (opcional). Si un equipo decide abordar aspectos adicionales no funcionales en su proyecto, incluir una nueva sección describiéndolos en apartados separados.
6. **Conclusiones**.
   - Análisis crítico, en retrospectiva, sobre el desarrollo del proyecto: ¿Qué aspectos fueron los más difíciles de llevar a cabo? ¿Cómo se resolvieron? ¿Qué aspectos fueron los más sencillos de realizar? ¿Cual ha sido la experiencia del equipo en cuanto al desarrollo colaborativo propuesto?.
   - Conclusiones y experiencia personal del equipo.

Se recomienda que el informe final no exceda las 70 páginas.

Referencias

[1] MIKE COHN, *User stories applied for agile software development*, Boston, MA: Addison-Wesley, cop. 2004. Biblioteca UJI: QA76.76.D47 C64286 2004.

[2] ÓSCAR BELMONTE FERNÁNDEZ, CARLOS GRANELL CANUT y MARÍA DEL CARMEN ERDOZAIN NAVARRO, *Desarrollo de proyectos informáticos con tecnología Java*, 2012.

[3] CARLOS BLÉ JURADO, *Diseño Ágil con TDD*, Segunda Edición, 2020.